

Slick-Compress — Project Technical Guide

Project: Slick-Compress (lossless-compressor)

Type: Full-stack Next.js web application

Purpose: Format-aware file compression targeting ~40% savings with browser and native engines.

1. What This Project Does

Slick-Compress is a smart compression platform that picks the right algorithm per file type instead of using one generic ZIP pass. It includes:

- A marketing site (Home, Capabilities, How it works, About, Downloads)
 - A compression workbench (upload, choose goal/mode, compress, download)
 - User authentication (sign up, log in, log out)
 - Server-side compression for heavy formats (video, PDF, images via native tools)
 - Browser-side compression for text, images (lossy), and lossless algorithms
-

2. Technology Stack

Layer	Technology	Role
Framework	Next.js 16 (App Router)	Pages, routing, Server Actions
UI	React 19, Tailwind CSS 4	Components and styling
Language	TypeScript	Type-safe app and library code
Auth	Node crypto (scrypt) + HTTP-only cookies	Local user store in <code>.data/users.json</code>
Build	Webpack (via <code>next dev/build --webpack</code>)	Bundling (Turbopack disabled for <code>wasm/ffmpeg</code>)

3. NPM Libraries — What Each One Does

Runtime dependencies (used in the app)

Package	Used for
next	App framework, Server Actions, API routes
react / react-dom	UI components
brotili-wasm	Brotli compression/decompression in the browser (WASM)
fflate	Gzip/deflate in the browser (<code>gzipSync</code> , <code>gunzipSync</code>)
pako	Additional deflate support (if referenced)
browser-image-compression	Optional image helpers
jxl-wasm	JPEG XL WASM support (available for future image paths)
@ffmpeg/ffmpeg + @ffmpeg/util	In-browser video transcoding fallback (ffmpeg.wasm)

Dev dependencies (build / docs)

Package	Used for
docx	Generates Word project slides (<code>generate-deck.mjs</code>)
pptxgenjs	Legacy slide generation (superseded by template merge for PPTX)
sharp	Snapshot image processing in deck scripts
tailwindcss	Utility CSS
eslint / eslint-config-next	Linting

Native tools (not npm — must be installed on the machine)

Tool	Install	Used for
ffmpeg	<code>brew install ffmpeg</code>	Video H.264/H.265 transcoding, WebP/JPEG image passes
ffprobe	(bundled with ffmpeg)	Validates encoded video before download
ghostscript (<code>gs</code>)	<code>brew install ghostscript</code>	Lossy PDF recompression (30–40% on image-heavy PDFs)
sips	macOS built-in	JPEG quality passes on macOS when ffmpeg unavailable

4. Compression Goals & Modes

Compression goals (user selects in workbench)

Goal	Where it runs	Behavior
Target 40% (recommended)	Browser + server for PDF	Lossy when needed (~40% target). PDFs use Ghostscript on server. Images use WebP/JPEG sweeps. Other files use best lossless in browser.
High impact local	Server (native binaries)	ffmpeg for video, Ghostscript for PDF, sips/ffmpeg for images, Brotli/Gzip for text/office
Strict lossless	Browser	Brotli, Gzip, RLE, LZW — exact byte restore

Compression mode (Fast / Balanced / Maximum)

- Affects **browser** lossless quality steps and **image** quality sweeps
- For **PDF**, lossy runs always use **Maximum** Ghostscript profiles when targeting 40% savings
- For **video**, affects ffmpeg preset/CRF style via `video-ffmpeg-args.ts`

Optimization profile (Smart / Text / Media)

- **Smart:** All algorithms as appropriate
 - **Text:** Favors Brotli/RLE/LZW; skips heavy media assumptions
 - **Media:** Accept hints for video/audio/image uploads
-

5. What Works for What — Format Matrix

File type	Best goal	Engine	Method	Typical savings
MP4, MOV, MKV, AVI	High impact local	ffmpeg (server)	H.264 + H.265 (hvc1), pick smallest	40–80%
Video (no ffmpeg)	High impact local	ffmpeg.wasm (browser)	H.264 + AAC fallback	Varies
PDF	Target 40% or High impact local	Ghostscript (server)	Lossy image downsample + JPEG re-encode	30–40% (image-heavy); less for text-only
JPEG / PNG / WebP	Target 40%	Canvas (browser)	WebP/JPEG quality + scale sweeps	35–60%
CSV, TXT, JSON, XML, MD	Any / High impact local	Brotli (browser or Node zlib)	Brotli Q11	20–90%
DOCX, PPTX, XLSX, etc.	High impact local	Node zlib	Brotli or Gzip on raw bytes	20–90%
Pre-compressed .br / .gz	Any	Decompress then recompress inner content	Wrapper unpack first	Depends on inner file

Why PDF was only ~4% before

- Browser **lossless** path (Brotli/Gzip on PDF bytes) barely shrinks already-compressed PDF streams.
- Fix: route PDFs to **Ghostscript lossy** with `PassThroughJPEGImages=false` and aggressive DPI/JPEG quality.

6. Project File Structure — What Each File Does

`app/` — Next.js pages and server actions

File	Purpose
app/layout.tsx	Root layout: fonts, AuthProvider , header, footer
app/page.tsx	Home / landing page
app/compress/page.tsx	Compression workbench route
app/capabilities/page.tsx	Capabilities marketing page
app/how-it-works/page.tsx	Pipeline explanation
app/about/page.tsx	About the project
app/downloads/page.tsx	Download slides + technical guide PDF
app/login/page.tsx	Login form
app/signup/page.tsx	Sign up form
app/actions.ts	Server Action compressNativeAction — ffmpeg, Ghostscript, Brotli, office
app/actions/auth.ts	Server Actions: signup, login, logout, getCurrentUser
app/globals.css	Global styles, progress animation
next.config.ts	Webpack, 100MB Server Action body limit

components/ — React UI

File	Purpose
components/compress-workbench.tsx	Main upload UI, progress %, dispatches browser vs server compression
components/site-header.tsx	Navigation, auth links, logout
components/site-footer.tsx	Footer
components/page-shell.tsx	Page width/padding wrapper
components/auth-provider.tsx	React context for logged-in user
components/auth-form.tsx	Login/signup forms with Server Actions

lib/ — Core logic

File	Purpose
<code>lib/compression-engine.ts</code>	Browser compression: Brotli, Gzip, RLE, LZW, image sweeps, ffmpeg.wasm video
<code>lib/pdf-ghostscript.ts</code>	Ghostscript CLI argument profiles (lossy PDF)
<code>lib/pdf-compress-server.ts</code>	Runs multiple GS profiles, picks smallest, optional 2nd pass
<code>lib/video-ffmpeg-args.ts</code>	QuickTime-safe ffmpeg arguments (H.264/H.265 hvc1)
<code>lib/resolve-binary.ts</code>	Finds <code>ffmpeg</code> / <code>gs</code> in PATH (Homebrew, etc.)
<code>lib/auth-server.ts</code>	User store, password hashing, session cookies
<code>lib/site-content.ts</code>	Marketing copy, nav links, capabilities text

`scripts/` — CLI utilities

File	Purpose
<code>scripts/generate-deck.mjs</code>	Builds PPTX (from internship template) + DOCX slides
<code>scripts/merge-internship-template.py</code>	Merges <code>Internship_Final.pptx</code> layout with project content
<code>scripts/style-pptx-borders-fonts.py</code>	Page borders + Times New Roman on deck
<code>scripts/generate-snapshots.mjs</code>	UI snapshot PNGs for slides
<code>scripts/generate-project-guide.mjs</code>	This technical guide → PDF
<code>scripts/compress-local.mjs</code>	CLI compress without the web UI

`public/` — Static assets

Path	Purpose
<code>public/downloads/</code>	Generated slides, technical guide PDF
<code>public/downloads/snapshots/</code>	Slide screenshot images
<code>public/downloads/templates/</code>	Internship PPTX template
<code>public/generated/</code>	Temporary compressed outputs from server (download URLs)
<code>public/slides.html</code>	Simple HTML slide deck (optional)

Data / temp (gitignored)

Path	Purpose
<code>.data/users.json</code>	Registered users (auth)
<code>.tmp-compress/</code>	Server-side temp files during compression

7. Compression Flow (How Code Runs)

Path A — Browser (`compress-workbench` → `compression-engine.ts`)

1. User selects file and goal (not PDF/video native path).
2. File read as `Uint8Array`.
3. `bestLosslessCompression()` tries Gzip → Brotli → RLE → LZW; picks smallest.
4. For images + Target 40%: `compressImageTowardTarget()` sweeps WebP/JPEG quality.
5. Result blob → download via object URL.

Path B — Server (`compress-workbench` → `compressNativeAction` in `actions.ts`)

1. FormData uploaded to Server Action (up to 100MB).
2. File written to `.tmp-compress/`.
3. Extension detected; routed to handler:
 - Video → ffmpeg H.264 + H.265, ffprobe validate, smallest wins
 - PDF → `compressPdfAggressive()` in `pdf-compress-server.ts`
 - JPEG → sips or ffmpeg quality passes
 - PNG/etc. → ffmpeg WebP sweep
 - Text/office → Node Brotli/Gzip
4. Output copied to `public/generated/` ; URL returned to client.

Path C — PDF specifically (lossy)

1. `pdfProfilesForMode()` builds Ghostscript argument sets.
 2. Each profile runs `gs -sDEVICE=pdfwrite` with downsampling + JPEG encode.
 3. Smallest valid PDF wins; if <30% saved, second pass with extreme profile.
 4. Key flags: `PassThroughJPEGImages=false` , low DPI, low `JPEGQ` .
-

8. Authentication

Piece	Location	Behavior
Sign up	<code>app/signup + signupAction</code>	Creates user in <code>.data/users.json</code> , sets session cookie
Log in	<code>app/login + loginAction</code>	Verifies bcrypt password hash
Log out	Header button + <code>logoutAction</code>	Clears <code>sc_session</code> cookie
Session	<code>lib/auth-server.ts</code>	HMAC-signed cookie, 7-day expiry

Set `AUTH_SECRET` in production `.env` for secure session signing.

9. Key Commands

```
|  
npm run dev           # Start dev server (localhost:3000)  
  
npm run build          # Production build  
  
npm run generate:deck  # Regenerate PPTX + DOCX slides  
  
npm run generate:guide # Regenerate this technical guide PDF  
  
npm run compress:local # CLI compression test  
|
```

Required for full features:

```
|  
brew install ffmpeg ghostscript  
|
```

10. Summary

Slick-Compress combines a **Next.js UI**, **browser WASM/JS codecs**, and **native CLI tools** (ffmpeg, Ghostscript) to compress different file types appropriately. The most important design choice is **format-aware routing**: video → ffmpeg, PDF → lossy Ghostscript, images → canvas lossy, text → Brotli — instead of one algorithm for everything.

Generated for the Slick-Compress / lossless-compressor repository.